

Dokumen Desain Arsitektur Pertarungan Bos: Subo, Sang Mutan Monopoli Argentara

1. Dekonstruksi Kegagalan Pertarungan Bos 2D dan Sintesis Solusi Struktural

Sejarah panjang dan evolusi genre aksi-platformer dua dimensi dipenuhi dengan berbagai ambisi desain pertarungan bos yang pada akhirnya gagal memberikan pengalaman psikologis dan mekanis yang memuaskan bagi para pemain. Dalam banyak literatur dan analisis desain permainan, kegagalan ini sering kali berakar pada ketidakmampuan perancang untuk menyelaraskan ekspektasi kognitif pemain dengan komunikasi visual yang dipancarkan oleh mesin permainan¹. Terdapat tiga kelemahan arsitektural fundamental yang secara sistemik merusak integritas pertarungan bos pada era klasik maupun modern, yang menuntut dekonstruksi analitis sebelum kita dapat merancang sebuah mahakarya yang mendefinisikan standar industri.

Kelemahan pertama adalah ketergantungan empiris pada arsitektur "Spons Peluru" (*Damage Sponge*). Mayoritas permainan platformer meningkatkan kurva tingkat kesulitan secara artifisial dengan menggandakan kapasitas poin kesehatan (*Health Points*) dari bos, yang pada dasarnya memaksa pemain untuk melakukan repetisi serangan tanpa akhir¹. Desain semacam ini secara langsung menghancurkan esensi pendorong motivasi inti (*Core Drive*) dari sebuah pencapaian, karena tantangan yang disajikan bertransformasi menjadi sebuah kebosanan berbasis atrisi alih-alih ujian keterampilan motorik dan spasial yang terkalibrasi secara elegan. Pertarungan bos seharusnya berfungsi sebagai "ruang kelas" ujian akhir, di mana bos menuntut pembuktian atas segala mekanik yang telah dipelajari pemain sepanjang level, bukan sekadar menghabiskan waktu mereka³.

Kelemahan kedua bermanifestasi pada ketiadaan telegraf serangan (*attack telegraphing*) yang adil secara kinestetik. Pada banyak permainan aksi yang kurang dipoles secara presisi, bos sering kali bertransisi dari state diam (*idle*) menuju state serangan aktif (*active hitbox*) dalam rentang 33 hingga 50 milidetik². Mengingat waktu reaksi rata-rata manusia berada di kisaran 200 hingga 250 milidetik, ketiadaan jendela persiapan (*wind-up*) ini membuat penghindaran berbasis refleks menjadi kemustahilan biologis. Kemenangan pemain kemudian direduksi menjadi probabilitas acak atau memaksa strategi pertukaran kerusakan (*damage trading*) yang menghilangkan estetika pertempuran.

Kelemahan ketiga berkaitan dengan representasi kotak tabrakan (*hitbox*) yang tidak akurat, di mana batas visual dari aset grafis (*sprite*) tidak kongruen dengan matriks kotak tabrakan logis yang dihitung oleh mesin deteksi *Axis-Aligned Bounding Box* (AABB)⁵. Kondisi ini melahirkan tragedi ruang di mana pemain menerima kerusakan fisik meskipun secara kasat mata avatar mereka tidak berpotongan dengan anatomi musuh, menciptakan frustrasi yang mendalam dan

melanggar kontrak kepercayaan antara perancang dan pemain.

Konteks permainan ini menetapkan hukuman yang sangat tinggi (*high-stakes penalty*); apabila Galuh tewas di Level 3, akumulasi poin dan progres direset secara total, memaksa pemain mengulang dari Level 1. Taruhan psikologis setinggi ini menuntut keadilan desain yang absolut, di mana setiap kegagalan harus dirasakan murni sebagai kesalahan pemain, bukan manipulasi sistem. Oleh karena itu, rancangan pertarungan melawan "Subo" menyintesiskan filosofi desain mekanis dari mahakarya seperti *Hollow Knight* dan seri klasik *Megaman*.

Sintesis struktural pertama adalah implementasi telegraf spasial dan visual yang absolut secara matematis. Setiap inisiasi serangan dari Subo akan diwajibkan melewati fase persiapan minimal 750 milidetik (0,75 detik), memberikan ruang bagi kognisi pemain untuk memproses ancaman⁷.

Sintesis kedua adalah integrasi ekonomi pertempuran (*Combat Economy*) yang proaktif.

Mengambil inspirasi dari sistem jiwa (*Soul*) pada *Hollow Knight* dan kearifan lokal *Jimpitan* (pengumpulan aset sedikit demi sedikit untuk mengatasi krisis komunal), sistem ini memanfaatkan "Perak" yang dikumpulkan dari tahap kuis dan platforming sebagai amunisi pendorong probabilitas kemenangan⁹. Sintesis ketiga adalah penerapan hukum fisika diskrit melalui "jus pertempuran" (*combat juice*) seperti *hitstop* (pembekuan layar sesaat) dan guncangan kamera ketika senjata Galuh membentur zirah kulit Subo, yang secara psikologis menerjemahkan bobot digital menjadi sensasi kinetis yang memuaskan¹¹.

2. Arsitektur Arena (Level Design)

Arena pertarungan berfungsi sebagai kanvas spasial di mana seluruh dinamika vektor X dan Y antara Galuh dan Subo direpresentasikan. Untuk memfasilitasi pertarungan yang berbasis penguasaan ruang tanpa anomali gesekan sudut (*AABB corner catching*), topologi arena dirancang dengan spesifikasi geometri Cartesius yang sangat terkontrol. Ruang operasi ini secara tematik dinamakan "Dermaga Fluktuasi Perak", merepresentasikan volatilitas pasar dan batas akhir antara lautan (habitat Sura) dan daratan muara (habitat Baya) di pesisir Surabaya. Batas absolut arena dirender pada resolusi layar penuh standar dengan lebar 1280 piksel dan tinggi 720 piksel, menciptakan rasio aspek 16:9⁸. Rasio panoramik ini krusial untuk memberikan ruang pandang periferal maksimal bagi pemain, yang mutlak diperlukan untuk mendeteksi telegraf serangan Subo yang sering kali berawal dari ujung batas layar. Lantai dasar merupakan bidang datar sempurna tanpa celah jebakan gravitasi (*pitfalls*). Penghapusan lubang jebakan adalah keputusan desain yang disengaja untuk memastikan bahwa kematian pemain seratus persen disebabkan oleh kegagalan membaca pola serangan Subo, mengeleminasi variabel kecelakaan platforming yang tidak terkait langsung dengan tarian pertempuran¹³.

Komponen Arena	Posisi Koordinat Absolut (X, Y)	Dimensi Lebar (px)
Lantai Dasar	X: 0 - 1280, Y: 640	1280
Bursa Atas Kiri	X: 150, Y: 450	200

Bursa Atas Tengah	X: 540, Y: 320	200
Bursa Atas Kanan	X: 930, Y: 450	200

Elevasi platform tidak ditempatkan secara acak, melainkan dikalibrasi secara langsung terhadap rumus kinematika lompatan karakter Galuh. Dengan ketinggian platform samping pada ordinat $Y = 450$ (190 piksel di atas permukaan lantai dasar), pemain tidak dapat mencapai platform ini dengan lintasan lompatan statis yang hanya mencapai tinggi maksimal 130 piksel¹⁵. Pemain harus memanfaatkan momentum lari dan penguasaan ujung kotak tabrakan (*edge of bounding box*) untuk meraih platform tersebut. Tata letak ini memecah arena menjadi berbagai zona risiko dan imbalan.

Platform tengah yang berada di ketinggian $Y = 320$ bertindak sebagai "Titik Buta Absolut" (*Absolute Blind Spot*) temporer untuk serangan horizontal yang melintasi lantai dasar. Namun, arena ini dilengkapi dengan sistem kecerdasan buatan anti-stagnasi. Berdiam diri terlalu lama di platform tengah (durasi lebih dari 2000 milidetik) akan memicu algoritma Subo untuk mendeteksi koordinat Y pemain secara konstan dan merespons dengan serangan anti-udara vertikal⁴. Arsitektur ini secara implisit memaksa ekosistem pergerakan pertempuran yang mengalir; sebuah koreografi kelangsungan hidup di mana tidak ada satu titik pun di layar yang menawarkan keamanan permanen, mengharuskan pemain untuk terus beradaptasi dengan siklus fluktuasi ancaman.

3. Mekanisme Player (Galuh) & Manajemen Aset

Karakter Galuh dirancang bukan sebagai entitas super yang kebal terhadap serangan, melainkan sebagai manifestasi dari manajemen risiko finansial dan presisi aksi. Secara tematik, ia adalah pahlawan yang mewarisi kearifan budaya melalui penggunaan Keris, dipadukan dengan kecerdasan literasi aset (Warta dan Perak). Seluruh perpindahan fisika tubuhnya dikalkulasi dengan mesin AABB (*Axis-Aligned Bounding Box*) yang terintegrasi pada skema waktu delta (*deltaTime*) untuk menghilangkan anomali *frame rate* pada berbagai perangkat penjelajah web, dengan perhitungan metrik yang dinormalisasi pada basis per-detik¹⁶.

Kinetika pergerakan fundamental Galuh direkayasa dengan tingkat kecepatan dan gravitasi yang merangsang kecekatan refleks. Kotak tabrakan Galuh dikunci pada dimensi yang merepresentasikan kepadatan piksel bagian tengahnya, yakni lebar 48 piksel dan tinggi 64 piksel, memberikan kelonggaran gesekan ruang (*grace pixel*) terhadap serangan musuh¹⁵. Kecepatan translasi horizontal ditetapkan secara statis pada 360 piksel per detik, sementara intervensi gravitasi ditarik pada besaran 1440 piksel per detik kuadrat. Untuk melakukan manuver vertikal, gaya dorong awal (*initial velocity*) dipatok pada -600 piksel per detik, yang secara simulatif menciptakan parabola lompatan berdurasi 417 milidetik hingga mencapai puncak kelembaman (*apex*) setinggi 125 hingga 130 piksel sebelum gravitasi menariknya kembali¹⁵.

Senjata utama Galuh, Keris peninggalan, memproyeksikan perluasan AABB ke arah depan

sebagai instrumen serangan dasar. Teknik ini secara kultural dan mekanis dinamakan "Sabetan Warta", mencerminkan tindakan penyebaran literasi melalui ketajaman analisis. Serangan ini tidak menggunakan tembakan peluru (*projectile*) untuk memaksa pemain memelihara kedekatan jarak (*spacing*) yang berbahaya dengan bos⁹.

Fase Keris (Sabetan Warta)	Durasi Temporal	Properti Mekanis AABB
Startup (Antisipasi)	67 ms	Waktu dari penekanan <i>input</i> hingga instansiasi matriks AABB Keris. Cepat namun membutuhkan komitmen kinestetik.
Active (Osilasi Mematikan)	100 ms	Kotak tabrakan mematikan dengan dimensi 40px (lebar) x 20px (tinggi). Modul perhitungan kerusakan memproyeksikan 10 HP.
Recovery (Penarikan)	200 ms	State penguncian kontrol. Pemain tidak dapat bergerak atau menghindari, menciptakan celah hukum retribusi jika ayunan meleset.

Sistem ekonomi tempur adalah mahkota filosofis dari perancangan karakter Galuh, menjembatani tema edukasi finansial Argentara dengan utilitas mekanis permainan. Mata uang "Perak" yang dikumpulkan dari Level 1 dan 2 bersifat deterministik; mewakili kelangkaan aset komoditas di dunia nyata tanpa batas dompet maksimum (*max wallet cap*). Perak ini tidak sekadar menjadi skor statis, melainkan sumber daya pertempuran dinamis (*active resource pool*)³. Hal ini memperkenalkan sistem manajemen kekayaan tempur; pemain yang mengumpulkan kekayaan secara maksimal akan memiliki keleluasaan manuver strategis di fase puncak.

Mekanisme pertama dari ekonomi perak ini adalah teknik "Suntikan Likuiditas Jimpitan" (Manuver *Invulnerability Dash*). Terminologi ini menyerap nilai budaya *Jimpitan* Surabaya, di mana penyisihan aset kecil digunakan sebagai jaring pengaman sosial saat krisis¹⁰. Dengan mengonsumsi 20 Perak (biaya absolut), Galuh mampu meledakkan energi yang mentranslasikan posisi fisiknya sejauh 180 piksel ke depan dalam kurun waktu singkat 167

milidetik. Selama jendela waktu ini, matriks deteksi tabrakan tubuh Galuh terhadap *hitbox* musuh dimatikan (*isInvulnerable = true*), memungkinkannya menembus serangan solid secara frontal. Namun, sistem AABB lingkungan tetap mengkalkulasi dinding untuk mencegah eksploitasi peretasan ruang (*out-of-bounds clipping*).

Mekanisme eskalasi kedua disebut "Tebasan Dividen Suroboyo". Ini adalah kemampuan penembusan zirah mutlak yang menuntut pengeluaran biaya makro sebesar 50 Perak (biaya absolut). Galuh menyalurkan fluktuasi perak ke bilah Keris, menciptakan AABB masif (lebar 80 piksel) yang memproyeksikan kerusakan setara 40 HP. Secara fungsional, serangan ini dirancang sebagai instrumen solusi sistemik (*hard counter*) untuk memecahkan blokade perisai Kulit Buaya Subo dari arah frontal, yang juga menyuntikkan status henti paksa (*stagger state*) pada mesin logika bos selama 500 milidetik, memberi ruang bernapas bagi pemain.

4. Mekanisme BOSS (Subo) - Fase & Pola Serangan

Subo adalah antitesis dari stabilitas; sebuah monster mutan amorfosis yang mengawinkan kelincahan agresif Hiu (Sura) dan ketangguhan absolut Buaya (Baya). Representasi makhluk ini merupakan metafora dari keganasan siklus pasar yang siap menghancurkan entitas tanpa strategi yang matang. Matriks spasial Subo dikonstruksi dengan skala raksasa yang mendominasi piksel layar, berdimensi lebar 160 piksel dan tinggi 192 piksel¹⁵. Besaran *hitbox* ini dirancang untuk menyulitkan navigasi pemain, menuntut penggunaan teknik mobilitas platforming yang sempurna.

Pertarungan melawan Subo didekonstruksi menjadi tiga fase operasional yang ditransisikan secara mulus tanpa menghentikan kelajuan pertempuran, dipicu oleh ambang batas dari total kapasitas kesehatannya (1000 HP). Pendekatan transisi dinamis ini meminimalisir interupsi psikologis (*flow interruption*) dan memaksa kurva pembelajaran yang adaptif¹⁹.

Terminologi Fase Subo	Ambang Batas Logika HP	Karakteristik Mesin Kecerdasan Buatan (AI)
Fase 1: Agresi Pasar	1000 - 660 HP	Dominasi pergerakan lateral hiu; jangkauan serangan horizontal ekstensif.
Fase 2: Monopoli Buaya	659 - 330 HP	Aktivasi <i>armor</i> kulit; introduksi serangan cakar anti-udara dan penolakan kinetis.
Fase 3: Krisis Argentara	329 - 0 HP	Hibridisasi serangan berskala area (AoE) dan penyusutan durasi pemulihan bos.

Pada Fase 1, Subo mengandalkan karakteristik hiunya (Sura) dengan mobilitas tinggi yang merobek penguasaan ruang lantai arena. Serangan primernya, "Sabetan Fluktuasi", merupakan sapuan ekor yang sangat mematikan namun tertelegraf dengan adil². Sebelum eksekusi, Subo memasuki masa transisi persiapan selama 750 milidetik, merendahkan titik beratnya dengan animasi matanya yang menyala berkedip. Menjelang sabetan, geometri hitung ekornya membengkak secara dinamis dari lebar 160 piksel menjadi 320 piksel, dengan batas tinggi 60 piksel dari lantai dasar. Lintasan destruktif ini mengharuskan Galuh untuk mengandalkan kinematika presisi dari lompatannya, memastikan ketinggiannya melewati ambang 60 piksel tepat di kala sapuan menyapu lantai.

Masih di fase yang sama, Subo sering mengeksekusi "Terjangan Bearish" (*Forward Charge*), sebuah lari serudukan yang menggambarkan tren kejatuhan brutal. Setelah jeda vokal dan efek partikel selama 667 milidetik, algoritma jarak akan mengunci posisi dan Subo melesat membelah 800 piksel jarak kanvas pada kecepatan 960 piksel per detik (melintasi arena dalam ~0,83 detik)¹⁵. Kecepatan ini mendikte pemain untuk menggunakan platform Bursa Atas atau melakukan manuver *wall-jump*. Setelah terjangan, Subo tertahan dalam durasi pemulihan pasca-serangan (Recovery) selama 1000 milidetik penuh⁴. Pada momentum lelah inilah satu-satunya jendela titik buta (*blind spot*) absolut terbuka lebar di bagian ekor punggungnya untuk menerima rentetan Sabetan Warta dari Galuh.

Memasuki Fase 2, persentase kerusakan memicu transformasi biologis Subo. Dengan raungan yang mendistorsi getaran lensa layar kamera (*screen shake*), sisik buayanya (Baya) menegang, mengaktifkan modifikator state "Kulit Keras Monopoli". Arsitektur pertahanan ini membalikkan dinamika permusuhan secara drastis. Apabila Galuh mencoba meretas jarak dekat dan mendaratkan Sabetan Warta secara frontal (di mana orientasi pergerakan Keris bertabrakan tepat dengan orientasi vektor menghadap Subo), maka matriks mesin akan memveto kalkulasi kerusakan⁹. Serangan akan memantul (*Deflection*), mengkalkulasi pengurangan kerusakan ke angka nol, menendang Galuh dengan gaya pantul (*knockback recoil*) 50 piksel mundur, dan merender pembekuan skala waktu sementara selama 100 milidetik untuk mengamplifikasi tekstur berat pukulan yang gagal¹². Solusi mekanis untuk blokade ini mengharuskan evolusi strategi spasial: Galuh harus mencari titik buta di belakang punggung melalui lompatan akrobatik atau mencairkan aset 50 Perak demi "Tebasan Dividen Suroboyo" yang menyayat langsung melewati batas kulit tersebut.

Pada fase ini pula Subo mengintegrasikan serangannya menggunakan cakar buaya melalui "Cengkraman Inflasi Baya" (*Anti-Air Grab*). Ketika AI memonitor bahwa Galuh mengeksploitasi keamanan di platform Bursa Atas secara repetitif, Subo akan mengalihkan tatapannya 45 derajat ke arah langit-langit selama 500 milidetik sebelum meluncurkan rahang dan cakarnya menembus platform atas dalam lintasan parabola⁷. Strategi ini berfungsi ganda: memberikan hukuman struktural bagi pemain pasif dan memaksa pemanfaatan rotasi seluruh zona arena. Fase 3, "Krisis Argentara", merupakan fase keputusan algoritma (*desperation phase*). Parameter waktu jeda (*recovery*) Subo dipangkas sebanyak 30 persen, dan ia melepaskan hibridisasi "Tsunami Defisit". Subo melompat tepat ke titik episentrum arena ($X = 640$) dan memukul cakar ke bumi, mereplikasi gelombang seismik dari kedua sisi layar (lebar AABB

merambat secepat 600 piksel per detik)²³. Gelombang area (AoE) ini memaksa pemain mundur ke udara, namun secara logis, ini adalah jebakan berantai: sesaat setelah Galuh mendarat di platform, Subo langsung merangkaikannya dengan Cengkraman Inflasi Baya tanpa jeda normal. Skema brutal ini menguji kemampuan retensi pamungkas pemain, mengharuskan aktivasi pembakaran aset "Suntikan Likuiditas Jimpitan" (I-frames) untuk bergulir menembus gelombang defisit tanpa harus naik ke platform udara yang mematikan.

5. Logika Implementasi Vanilla JS (Untuk Coder)

Mendesain pertarungan mahakarya di atas kertas sangat berbeda dengan mengeksekusinya dalam bahasa komputasi murni. Penggunaan HTML5 Canvas dan sistem fisika JavaScript mentah (Vanilla JS) menuntut arsitektur logika yang sangat tangguh agar tidak membuahkan deteksi tembus ruang yang merusak imersi. Fondasi komputasional bos ini disangga oleh dua pilar rekayasa perangkat lunak: *Finite State Machine* (FSM) terskala dan Resolusi Tabrakan AABB Berfase (*Phased AABB Collision Resolution*)²³.

Pengorganisasian kecerdasan artifisial Subo dieksekusi melalui arsitektur hirarki *Finite State Machine* berbasis pemrograman berorientasi objek (OOP). Pendekatan primitif menggunakan rantai percabangan kondisi (*If-Else Hell*) sering kali gagal menjaga konsistensi state peralihan, menyebabkan animasi patah atau perhitungan kerusakan ganda²⁵. Dengan FSM, logika perilaku bos dikelompokkan menjadi modul-modul tertutup (*encapsulated states*). Mesin mendelegasikan perintah berdasarkan state aktif dan mengeksekusi tiga fase metodis: instruksi inisialisasi (`enter()`), siklus simulasi (`update(deltaTime)`), dan pembersihan sebelum beralih (`exit()`)²⁵.

Struktur FSM ini memantau secara mandiri persentase ambang batas kerusakan. Sebagai contoh, di dalam blok `IdleState.update(deltaTime)`, sistem akan terus memindai posisi Galuh. Ketika algoritma memutuskan untuk mengeksekusi serangan serudukan dan syarat *cooldown* terpenuhi, state bertransisi memanggil `changeState('TELEGRAPHING_DASH')`. Setelah durasi matematis 667 milidetik terpenuhi secara presisi melalui akumulasi `deltaTime`, status dialihkan seketika ke `ATTACK_DASH`, mengalokasikan parameter fisika kecepatan ekstrem (960 piksel per detik) tanpa residu perhitungan logika statik sebelumnya²⁷.

Dalam ranah kalkulasi geometri, deteksi matriks tabrakan *Axis-Aligned Bounding Box* menguraikan validasi potong ruang antara kanvas Subo dan Galuh. Algoritma Boolean sederhana untuk mengevaluasi silang sumbu koordinat didefinisikan melalui ketetapan komparasi ekstremitas kotak: perpotongan terjadi jika dan hanya jika seluruh proyeksi X dan Y antara objek A dan objek B saling tumpang tindih secara absolut⁵.

Namun, pergeseran ekstrim yang dieksekusi Subo pada fase "Terjangan Bearish" memicu masalah mendasar dalam fisika komputer spasial yakni anomali terowongan (*Tunneling Effect*). Pada kecepatan 960 piksel per detik, Subo berpotensi melompati seluruh lebar badan Galuh (yang hanya berukuran 48 piksel) dalam satu siklus `deltaTime`, gagal mendaftarkan tumbukan yang seharusnya terjadi di ruang antar-frame¹⁴. Untuk mengeliminasi probabilitas matematis ini, logika AABB diinjeksi dengan sistem *Swept AABB* yang berfungsi sebagai fase pendeteksian luas (*Broad-Phase Collision*). Mesin tidak sekadar mengecek posisi saat ini, melainkan menarik prisma ruang berdasarkan vektor lintasan masa depan Subo melalui parameter waktu delta,

mengkalkulasi prediksi waktu benturan parsial (*Time of Impact*), lalu menghentikan translasi ruang Subo tepat sebelum terjadinya tembus batas objek fisik sekitarnya²⁵.

Menanggapi kebutuhan estetis dari Sabetan Fluktuasi, mesin Vanilla JS diperkaya dengan sistem penyesuaian skala *hitbox* dinamis (*Dynamic Hitbox Scaling*). Kotak mematikan ekor hiu tidak dirender statis menyatu dengan batas tubuh utamanya. Melalui variabel sinkronisasi waktu animasi, algoritma menyuntikkan instruksi pengubahan manipulasi lebar kotak *hitbox* dari matriks dasar 160 piksel memuai menjadi ekstensi 320 piksel hanya di antara rentang waktu milidetik ke-750 hingga 917 dari awal mula serangan, menyapu persis di ruang visual yang direpresentasikan aset animasi sebelum mengerut kembali³⁴.

Sebagai klimaks penambahan kedalaman umpan balik haptik, logika pertempuran mentransformasikan waktu melalui implementasi *Hitstop* (pembekuan bingkai). Fitur esensial dalam tata letak "jus permainan" ini (*game juice*) memanipulasi persepsi dampak kinetik ketika hantaman Keris Galuh ditolak oleh modifikator Kulit Buaya¹¹. Berbeda dengan metode pemberhentian perulangan sistem yang dapat merusak siklus `requestAnimationFrame` peramban web (*browser*), arsitektur kustom di sini memotong utilitas penggerak waktu delta¹⁶. Variabel pengunci global, `hitStopTimer`, diinjeksikan dengan kompensasi waktu yang mengkarantina pemanggilan perhitungan gerak fisika. Selama interval mikro 100 milidetik yang dramatis ini, peramban tetap menjalankan sirkulasi instruksi pelukisan matriks `render()`, sehingga mesin grafis dapat memuntahkan percikan partikel logam dan mengguncang sistem pemosisi lensa kamera, sementara entitas karakter terkunci statis dalam ruang.

6. Variabel Konfigurasi Statis (Untuk AI Coder)

Bagian ini merangkum seluruh parameter operasional numerik ke dalam struktur pseudo-code / JSON, untuk diimplementasikan langsung sebagai variabel konstanta di dalam mesin JavaScript.

```
JavaScript
const CONFIG = {
  // 1. Arsitektur Arena
  ARENA: {
    WIDTH 1280 // Lebar total kanvas (piksel)
    HEIGHT 720 // Tinggi total kanvas (piksel)
    GROUND_Y 640 // Posisi absolut lantai dasar pada koordinat Y
  },
  PLATFORMS: [
    LEFT X 150 Y 450 WIDTH 200
    CENTER X 540 Y 320 WIDTH 200
  ]
}
```


RIGHT X 930 Y 450 WIDTH 200

// 2. Fisika & Manajemen Aset Galuh (Player)

PLAYER

HITBOX WIDTH 48 HEIGHT 64

PHYSICS

SPEED_X 360 // Kecepatan horizontal (px/sec)

GRAVITY 1440 // Tarikan gravitasi (px/sec^2)

JUMP_VELOCITY 600 // Tenaga tolakan lompatan Y (px/sec)

ATTACK_BASIC // Sabetan Warta

DAMAGE 10

HITBOX WIDTH 40 HEIGHT 20

TIMING

STARTUP_MS 67

ACTIVE_MS 100

RECOVERY_MS 200

SKILLS

DASH // Suntikan Likuiditas Jimpitan

COST_SILVER 20

DISTANCE_PX 180

DURATION_MS 167

HEAVY // Tebasan Dividen Suroboyo

COST_SILVER 50

DAMAGE 40

HITBOX WIDTH 80 HEIGHT 20

STAGGER_BOSS_MS 500

// 3. Mekanisme & AI Subo (Boss)

BOSS

HITBOX WIDTH 160 HEIGHT 192

HP

MAX 1000

PHASE_2_THRESHOLD 659

PHASE_3_THRESHOLD 329

```

MECHANICS
CENTER_CAMP_LIMIT_MS 2000 // Waktu maksimal Galuh bisa diam di tengah sebelum memicu
Anti-Air
DEFLECTION
KNOCKBACK_PX 50
HITSTOP_MS 100
ATTACKS
TAIL_SWIPE // Sabetan Fluktuasi
TELEGRAPH_MS 750
HITBOX_EXPANDED WIDTH 320 HEIGHT 60
CHARGE // Terjangan Bearish
TELEGRAPH_MS 667
SPEED_X 960 // Kecepatan lari (px/sec)
RECOVERY_MS 1000
ANTI_AIR // Cengkraman Inflasi Baya
TELEGRAPH_MS 500
TSUNAMI // Tsunami Defisit (Area of Effect)
SPEED_X 600 // Kecepatan rambat ombak (px/sec)

```

Karya yang dikutip

1. Game Design Technique #14: Mastering the Boss Fight - Yu-kai Chou,
<https://yukaichou.com/advanced-gamification/game-design-technique-14-mastering-the-boss-fight/>
2. questioning boss designs and mechanics :: Hollow Knight: Silksong General Discussions - Steam Community,
<https://steamcommunity.com/app/1030300/discussions/0/598537454201522666/>
3. Character Sheet Prototype.pdf
4. Boss Design and how to do Difficulty. (This post includes imagery of both Hollow Knight & Silksong bosses) : r/HollowKnight - Reddit,
https://www.reddit.com/r/HollowKnight/comments/10aec91/boss_design_and_how_to_do_difficulty_this_post/
5. 2D collision detection - Game development - MDN Web Docs,
https://developer.mozilla.org/en-US/docs/Games/Techniques/2D_collision_detection

6. Controlling the Accuracy and Efficiency of Collision Detection in 2d Games using Hitboxes,
https://www.researchgate.net/publication/374298154_Controlling_the_Accuracy_and_Efficiency_of_Collision_Detection_in_2d_Games_using_Hitboxes
7. Silksong Devs Explain How Bosses Were Designed Differently Than Those in Hollow Knight,
<https://www.military.com/off-duty/games/silksong-devs-explain-how-bosses-were-designed-differently-those-hollow-knight.html>
8. [unknown_url](#)
9. The Hypnotizing Simplicity of Hollow Knight's Combat | by Rory Hoeschen - Medium,
<https://medium.com/@roryhoeschen/the-hypnotizing-simplicity-of-hollow-knight-s-combat-a60f820fd433>
10. Jimpitan: Indonesia's Local Wisdom - Universitas Airlangga Official Website,
<https://unair.ac.id/en/jimpitan-indonesias-local-wisdom/>
11. Freeze Frame/hitstop in action games - why? Can it be done better? : r/truегaming - Reddit,
https://www.reddit.com/r/truегaming/comments/xk8hth/freeze_framehitstop_in_action_games_why_can_it_be/
12. How can I make the charge punch feel more impactful? : r/godot - Reddit,
https://www.reddit.com/r/godot/comments/1f1324c/how_can_i_make_the_charge_punch_feel_more/
13. Basic 2D Platformer Physics, Part 7: Slopes Groundwork | Envato Tuts+,
<https://code.tutsplus.com/basic-2d-platformer-physics-part-7-slopes-groundwork--cms-28472t>
14. Swept AABB Collision Detection Conflict with Tile-sized Gaps,
<https://gamedev.stackexchange.com/questions/185664/swept-aabb-collision-detection-conflict-with-tile-sized-gaps>
15. 2D Platformer Boss "Code Design" Discussion - Epic Developer Community Forums,
<https://forums.unrealengine.com/t/2d-platformer-boss-code-design-discussion/109933>
16. Game Engine - Awesome GitHub Copilot,
<https://awesome-copilot.github.com/skill/game-engine/>
17. Building a Browser Game Where You Fly Through a Forest (It's Easier Than You Think),
<https://dev.to/alanwest/building-a-browser-game-where-you-fly-through-a-forest-its-easier-than-you-think-45hp>
18. 2D Platform Games Part 1: Collision Detection for Dummies,
<https://katyscode.wordpress.com/2013/01/18/2d-platform-games-collision-detection-for-dummies/>
19. A Programming Model for Boss Encounters in 2D Action Games - Association for the Advancement of Artificial Intelligence (AAAI),
<https://cdn.aaai.org/ojs/12891/12891-52-16408-1-2-20201228.pdf>
20. Designing a Boss: Part 2 -Using a State Machine | by Dustin Cargile - Medium,

<https://medium.com/@dcargile84/designing-a-boss-part-2-using-a-state-machine-3d04a4700890>

21. How to properly do a freeze-frame / hit-stun, conceptually ? : r/gamedev - Reddit, https://www.reddit.com/r/gamedev/comments/5dfdr6/how_to_properly_do_a_freezeframe_hitstun/
22. Hitstop - Fighting Games secret Juice [GameMaker | Vfx] - YouTube, <https://www.youtube.com/watch?v=6FRUg-Lu99M>
23. Collision Detection in 2D and 3D: Concepts and Techniques - GarageFarm, <https://garagefarm.net/blog/collision-detection-in-2d-and-3d-concepts-and-techniques>
24. Javascript Game Foundations - State Management - Jake Gordon, <https://jakesgordon.com/writing/javascript-game-foundations-state-management/>
25. Beyond If-Else Hell: Elegant State Machine pattern in Game Development - DEV Community, https://dev.to/niraj_gaming/beyond-if-else-hell-elegant-state-machines-pattern-in-game-development-2i7g
26. a simple collision resolution algorithm for 2D action games - Charlie, <https://c.har.li/e/2024/03/28/implementing-robust-2D-collision-resolution.html>
27. An introduction to state machines and the state pattern - The Shaggy Dev, <https://shaggydev.com/2021/11/01/state-machines-intro/>
28. Pong - The Game Loop | Jake Gordon, <https://jakesgordon.com/writing/javascript-pong/part3/>
29. adamfk/game-fsm-examples: game development state machine (StateSmith) examples - GitHub, <https://github.com/adamfk/game-fsm-examples>
30. 3D collision detection - Game development - MDN Web Docs, https://developer.mozilla.org/en-US/docs/Games/Techniques/3D_collision_detection
31. The Math Behind Bounding Box Collision Detection - AABB vs OBB(Separate Axis Theorem), https://dev.to/pratyush_mohanty_6b8f2749/the-math-behind-bounding-box-collision-detection-aabb-vs-obbsseparate-axis-theorem-1gdn
32. Swept AABB collision between circle and rectangle - Stack Overflow, <https://stackoverflow.com/questions/79650138/swept-aabb-collision-between-circle-and-rectangle>
33. AABB collision resolution slipping sides - Stack Overflow, <https://stackoverflow.com/questions/46172953/aabb-collision-resolution-slipping-sides>
34. Animation Hitboxes · Documentation: Replicated Hitbox System - Coda, <https://coda.io/@aurora-devs/documentation-replicated-hitbox-system/animation-hitboxes-4>
35. 2D Hitboxes/Hurtboxes Based on Frame - Unity Discussions, <https://discussions.unity.com/t/2d-hitboxes-hurtboxes-based-on-frame/621525>
36. Setting Up a Hitbox Attack System in Unity2D (1): Animated BoxCollider2D and "Attack" Script - Marcus Ansley,

<https://m-ansley.medium.com/setting-up-a-hitbox-attack-system-in-unity2d-1-animated-boxcollider2d-and-attack-script-cc97fcd3b268>

37. Hello, I am looking to recreate a Hit Stop effect similar to how you see in fighting games.,

<https://discussions.unity.com/t/hello-i-am-looking-to-recreate-a-hit-stop-effect-similar-to-how-you-see-in-fighting-games/910407>